

Las invariantes ponen restricción y mantienen la coherencia, como así también no van a mencionar a las funciones de la interfaz, solamente hablan de la estructura de esta, se enfoca solamente en la estructura, NO DEBE MENCIONAR FUNCIONES!!!. Las invariantes de representación son atemporales, no se que paso antes, ni se que va a pasar después. Las invariantes de representación son las restricciones sobre el tipo de representación (como es la estructura). Como identifico que es y que no es una invariante? cuando se puede hacer un caso que no se cumpla y un caso que se cumpla.

data RV = MkRV (Map CUIF Foto) (Map Vehiculo (Set Vehiculo))

INV. REP.: Siendo MkRV mcf mvv

- * Por cada clave c en mcf su valor asociado Vehiculo debe tener como CUIF dicha clave c.

- * Si un vehiculo aparece en alguna foto de mcf, debe ser clave en mvv.

- * Si un vehiculo v1 aparece con otro v2, entonces v2 también aparece con v1.

- * Ninguna clave c en mcf aparece dentro de su valor asociado Set Vehiculo.

data Nave = N (Map SectorId Sector) (Map Nombre Tripulante) (MaxHeap Tripulante)

INV. REP.: Siendo N MapS MapT HeapT

- * Para cada clave i en Map S, el Sector asociado debe tener id i

- * Para cada clave t en Map T, el Tripulante asociado debe tener nombre i

- * Para cada valor asociado t en Map T, los sectores asignados a t son claves de Map S y el sector asociado tiene a t como Tripulante

- * Para cada valor asociado s en el Map S, los tripulantes asignados a s son claves de Map T y el tripulante asociado tiene a s como Sector Asginado

- * Para cada valor asociado t en el Map T, t debe pertenecer a la Heap T y viceversa

data RAList a = MkR Int (Map Int a) (Heap a)

INV. REP.: Siendo MkR n mia ha

- * El número n debe ser la cantidad de elementos en ha + 1.

- * Todos los valores asociados en mia deben estar en ha.

- * Todos los elementos de ha deben estar como valores asociados en mia.

- * El número n representa la cantidad de claves totales en msi + 1.

- * El número n no puede ser menor a 0 y tiene que ser entero.

data EscuelaDeMagia = EDM (Set Hechizo) (Map Nombre Mago) (PriorityQueue Mago)

INV. REP.:

* Para cada Nombre de Mago asociado en el Map hay un Mago con el mismo nombre en la PQ y viceversa

* Todos los hechizos de cada Mago estan en el Set de Hechizos

* En la PQ no hay dos magos con el mismo nombre

data Reality = R (Map Nombre Participante) (Map Juego [Nombre]) (PriorityQueue Participante) Int

INV. REP.: Siendo R mps mjs ps jn

* El valor P asociado a cada clave n en mps cumple que nombre p es n

* Si J es elemento de Juego P, siendo P un valor asociado a clave en mps entonces J es clave en mjs y nombre P pertenece al valor asociado al mismo.

* Las listas asociadas a cada clave J en mjs no tienen repetidos

* Para cada P que sea valor asociado a una clave mps, P esta en PS

* Para cada P que esta en PS, P es valor a una clave en mps

* En PS no hay elementos repetidos

* jn es la cantidad de claves en mps

data Busqueda = B (Map String (Map String Int))

INV. REP.: Siendo B mpa fs

* Todos los elementos de fs estan relacionados con mpa.

* Todas las claves p en mpa satisfacen todos los filtros presentes en fs

* Todos los nombres de productos "String" como clave en mpa deben tener asociado un Map String Int que tiene al menos una clave valor "Precio" y un número n que representa dicho precio.

data Empresa = ConsE (Map SectorId (Set Empleado)) (Map CUIL Empleado)

INV. REP.: Siendo ConsE MapS MapE

* Para cada clave c en Map E, el empleado asociado tiene CUIL c (solo se puede hacer porque la interfaz empleado permite averiguar el CUIL)

* Ningun CUIL de las claves es negativo

* Para cada empleado e, valor de Map E, pertenece por lo menos a un conjunto valor de Map S

* Para cada empleado e, perteneciente a un conjunto s valor de Map S, e es un valor de Map E asociado a un CUIL

* Para cada Clave S en Map S, todos los empleados en el conjunto asociado a S, tienen a S en la lista de sectores en que trabajan

* Para cada Sector s en sectores e, siendo e Empleado valor en MapE, s debe ser clave en MapS y e debe pertenecer al set asociado en MapS

* Para todo e Empleado que pertenece a un set que es valor en MapS, e debe existir como valor en MapE

* Si un empleado e esta asociado a un CUIL c en MapE, entonces cuil e es c

TADs:

Stack: Es una lista de elementos donde se cumple la propiedad LIFO, el ultimo que entra es el primero que sale

Queue: Es una lista de elementos donde se cumple la propiedad FIFO el primero que entra es el primero que sale, es decir, se encola a lo ultimo de la lista y se desencola por delante

Set: Es una lista de elementos sin repetidos y una cantidad de elementos que tiene la lista

Priority Queue: Es una lista donde el primero en salir es el minimo elemento (el de mayor prioridad)

Map: Es una lista de tuplas clave valor, puede ser un diccionario, se elimina mediante la clave

MultiSet: Es una lista mejor conocida como bag que puede contener repetidos con la particularidad que es una estructura map donde la clave es el valor y el valor es la cantidad de veces que aparece ese valor en la lista

---A partir de aca no son TADs, son formas de implementar TADs

BST: Es un Set con formato de arbol de busqueda, los elementos del brazo izquierdo son menores que la raiz y la derecha son mayores, asi sucesivamente en cada nodo. Para buscar un elemento especifico se compara en primera instancia con la raiz y ahi se decide para que lado ir, para insertar es lo mismo. Si se borra la raiz se debe balancear el arbol subiendo el maximo de la izquierda o el minimo de la derecha.

AVL: Para mantener un balance en un BST se necesita la invariante AVL la que representa que las diferencias de alturas entre el brazo izquierdo y el brazo derecho no pueden ser mayor a 1. Esto se controla al insertar o borrar un valor.

Heap: Es una PQ con formato de arbol HEAP, donde la raiz es el minimo elemento y por consecuente los siguiente nodos respetan esa particularidad, los de maximo valor son los ultimos valores, un arbol lleno es que todos los niveles estan completos, a excepcion del ultimo que puede estar completo de izquierda a derecha pero no debe contener agujeros, tiene un unico lugar donde borrar y donde insertar. Al insertar se debe comparar con los valores cercanos para ver a donde tiene que rotar.

Interfaces de TADs:

Set, siendo N la cantidad de elementos del conjunto:

`emptyS :: Set a O(1)`

`isEmptyS :: Set a -> Bool O(1)`

`addS :: a -> Set a -> Set a O(log N)`

`belongsS :: a -> Set a -> Bool O(log N)`

`deleteS :: Eq a => a -> Set a -> Set a O(log N)`

`unionS :: Set a -> Set a -> Set a O(N * log N)`

`sizeS :: Set a -> Int O(1)`

`setToList :: Set a -> [a] O(N)`

Queue, siendo Q la cantidad de elementos presentes en la cola:

`emptyQ :: Queue a O(1)`

`enqueue :: a -> Queue a -> Queue a O(1)`

`firstQ :: Queue a -> a O(1)`

`dequeue :: Queue a -> Queue a O(1)`

`isEmptyQ :: Queue a -> Bool O(1)`

Map, siendo K la cantidad de claves distintas en el map:

`emptyM :: Map k v O(1)`

`assocM :: k -> v -> Map k v -> Map k v O(log K)`

`lookupM :: k -> Map k v -> Maybe v O(log K)`

`deleteM :: k -> Map k v -> Map k v O(log K)`

`domM :: Map k v -> [k] O(K)`

`sizeM :: Map k v -> Int O(1)`

PriorityQueue, siendo P la cantidad de elementos en la estructura:

`emptyPQ :: PriorityQueue a O(1)`

`isEmptyPQ :: PriorityQueue a -> Bool O(1)`

`insertPQ :: Ord a => a -> PriorityQueue a -> PriorityQueue a O(log P)`

`maxPQ :: PriorityQueue a -> a O(1)`

`deleteMaxPQ :: Ord a => PriorityQueue a -> PriorityQueue a O(log P)`

MultiSet, siendo M y M' la cantidad de elementos distintos en los multisets:

`emptyMS :: MultiSet a O(1)`

`addMS :: Ord a => a -> MultiSet a -> MultiSet a O(log M)`

`ocurrencesMS :: Ord a => a -> MultiSet a -> Int O(log M)`

`unionMS :: Ord a => MultiSet a -> MultiSet a -> MultiSet a O(M + M')`

`intersectionMS :: Ord a => MultiSet a -> MultiSet a -> MultiSet a O(M + M')`

`multiSetToList :: MultiSet a -> [(Int,a)] O(M)`

Heap, siendo M la cantidad de elementos en la heap:

`emptyH :: Heap a O(1)`

`isEmptyH :: Heap a -> Bool O(1)`

`insertH :: a -> Heap a -> Heap a O(log M)`

`findMaxH :: Heap a -> a O(1)`

`deleteMaxH :: Heap a -> Heap a O(log M)`

MaxHeap, siendo M la cantidad de elementos en la heap:

`emptyH :: MaxHeap a O(1)`

`isEmptyH :: MaxHeap a -> Bool O(1)`

`insertH :: a -> MaxHeap a -> MaxHeap a O(log M)`

`maxH :: MaxHeap a -> a O(1)`

`deleteMaxH :: MaxHeap a -> MaxHeap a O(log M)`

Costos:

1-El costo constante $O(1)$ son operaciones simples,

2-El costo logaritmico $O(\log n)$ son operaciones sobre ramas de un arbol siendo n la cantidad de elementos del arbol, normalmente operaciones de busqueda

3-El costo lineal $O(n)$ son operaciones que conllevan operaciones constantes pero repetitivas sobre una estructura, por ejemplo recorrer listas

4-El costo “Eneloguene” $O(n \log n)$ son operaciones que se llevan a cabo sobre un arbol y que por cada rama se ocupa el costo $O(\log n)$. Es una combinacion de operaciones lineales $O(n)$ con las logaritmicas $O(\log n)$, en otras palabras depende de la cantidad de elementos a recorrer.

5-El costo cuadrático $O(n^2)$ son operaciones que se llevan a cabo con otras operaciones del tipo $O(n)$, por lo que se entiende un recorrido sobre lista que ademas por cada elemento que recorre aplica una operacion de costo $O(n)$, en otras palbras si a cada elemento le hago una lineal se convierte en cuadrática.

6-Cubica

Operaciones de costo (Siempre mirar el costo de cada operación y como interactúan entre si):

$O(C + \log S)$ = es eso mismo

$O(S + \log S) = O(S)$

$O(\log E) + O(S * (\log S + \log E)) \rightarrow \log E + S \log S + S \log E \rightarrow S \log S + S \log E \rightarrow S(\log S + \log E)$

n `elem` ns

Tuplas:

consolidar p [] = [(p,1)]

consolidar p ((p',k):pks) = if esElMismoProyecto p p'

then (p', k + 1) : pks

else (p', k) : consolidar p pks

agregarAmigoHeap :: Persona -> MaxHeap (Int,Persona) -> MaxHeap (Int,Persona)

agregarAmigoHeap p mh = if isEmptyMH mh

then insertMH (1,p) mh

else let (i,p') = findMaxH mh

in if p' == p

then insertMH (i+1,p) (deleteMaxMH mh)

else insertMH (i,p') (agregarAmigoHeap p (deleteMaxMH

mh))

terminaCerrado :: [(Evidencia,Nombre)] -> Investigacion -> Bool

terminaCerrado [] inv = casoCerrado inv

terminaCerrado ((e,n):ens) inv = terminaCerrado ens (ingresarEvidencia e n inv)

saltear :: Int -> RondaMate -> (RondaMate, [Persona])

saltear n (RM rp mp n) = let (persona,rondaNueva) = siguiente rp

in saltear (n-1) (RM (enqueue persona rondaNueva) mp n)

siguientesN :: Búsqueda -> Int -> [(String,Int)]

siguientesN _ 0 = []

siguientesN búsqueda n = case siguiente búsqueda of

(Just producto, nuevaBusqueda) -> producto : siguientesN

nuevaBusqueda (n-1)

(Nothing,_) -> []

Recorrido sobre estructura similar árbol:

caminoAlTesoro' :: Mapa -> Maybe [Dir]

caminoAlTesoro' (Fin cf) = if tieneTesoro cf

```

        then Just []
        else Nothing

caminoAlTesoro' (Bifurcacion cf m1 m2) = if tieneTesoro cf
    then Just []
    else
        case caminoAlTesoro' m1 of
            Just xs -> Just (Izq : xs)
            Nothing -> case caminoAlTesoro' m2 of
                Just xs -> Just (Der: xs)
                Nothing -> Nothing

```

```

insertarEnPos :: Int -> a -> Stack a -> Stack a

```

```

insertarEnPos 0 e st = push e st

```

```

insertarEnPos n e st = if isEmptySt st

```

```

    then emptySt

```

```

    else if n > 0

```

```

        then push (top st) (insertarEnPos (n-1) e (pop st))

```

```

    else push e st

```

Operaciones sobre heap:

```

modificarHeap :: MaxHeap Tripulante -> Tripulante -> MaxHeap Tripulante

```

```

modificarHeap h t = if isEmptyH h

```

```

    then emptyH

```

```

    else if nombre(maxH h) == nombre t

```

```

        then insertH t (deleteMaxH h)

```

```

    else let máximo = maxH h

```

```

        in insertH máximo (modificarHeap (deleteMaxH h) t)

```

```

transformarHeapALista :: Ord a => Heap a -> [a]

```

```

transformarHeapALista ha = if isEmptyH ha

```

```

    then []

```

```

    else findMin ha : transformarHeapALista(deleteMin ha)

```